Experiment 2: Git, GitHub, and Merge Conflict Management

1. Overview of the Experiment

The goal is to understand source code management using Git and GitHub, focusing on branching, pull requests, resolving merge conflicts, and issue tracking.

2. How to Execute From Scratch

- Set up a repository on GitHub.
- Clone it locally to your machine, make your initial branch, and add configuration files.
- Intentionally modify the exact same line of a text file across two distinct branches and attempt to merge them into main to trigger a conflict. Clean it up manually and push it back up.

3. Step-by-Step Command List

- git clone <github-repository-url>
  - **Explanation:** Downloads a copy of the target GitHub repository onto your local system directory.
- git status
  - **Explanation:** Displays the current state of your working directory and staging area, highlighting untracked or modified files.
- git checkout -b feature-1
  - **Explanation:** Creates a brand-new branch named feature-1 and immediately switches your context to it.
- echo "Hello from feature-1" > branch.txt
  - **Explanation:** Generates or updates a file called branch.txt with designated placeholder text.
- git add branch.txt
  - **Explanation:** Stages the changes made to branch.txt, preparing it to be bundled into the next commit snapshot.
- git commit -m "Add txt from feature-1"
  - **Explanation:** Saves your staged snapshot to the local project history tracking timeline with an informative summary message.
- git push origin feature-1
  - **Explanation:** Uploads the local branch tracking changes and commit history straight up to the remote GitHub platform repo.
- git checkout main
  - **Explanation:** Switches your active terminal directory context back to the central production branch (main).
- git pull
  - **Explanation:** Fetches updating edits from the remote cloud repository tracking server

and instantly merges them into your local branch copy.

- git merge conflict-branch-1
  - **Explanation:** Combines files and updates from conflict-branch-1 into your current active branch. If changes overlap lines, Git stops and signals a merge conflict to fix.

4. Viva Q&A

- **Q: What fundamentally creates a merge conflict in Git?**
  - **A:** It happens when two different branches modify the exact same line(s) in a file, or when one branch deletes a file that another branch is modifying, and you attempt to merge them together. Git cannot automatically determine which version is correct.
- **Q: How do you manually resolve a merge conflict?**
  - **A:** Open the conflicted file, locate the conflict markers (<<<<<<<, =======, >>>>>>>), choose which code block to keep, remove the markers, save the file, run git add, and finalize it with git commit.

—----------

Experiment 3: Working with Virtual Machines on Cloud Platforms

1. Overview of the Experiment

This experiment details provisioning, configuring, connecting to, and deleting Linux Virtual Machines inside public cloud platforms like AWS EC2.

2. How to Execute From Scratch

- Log in to your cloud ecosystem dashboard (such as AWS).
- Launch a free-tier virtual compute instance running an operating system like Ubuntu 24.04 LTS.
- Open the firewall security structures to allow traffic on port 22 (SSH) and port 80 (HTTP).
- Connect directly via terminal command line arguments, perform package system

updates, spin up an active Nginx platform engine instance, and clean up your cloud workspace when finished.

3. Step-by-Step Command List

- ssh -i private-key.pem ubuntu@<your-vm-public-ip>
  - **Explanation:** Starts a secure encrypted shell session to remotely connect to and administer your cloud-hosted VM using your private identity key file.
- whoami
  - **Explanation:** Returns the login name of the user currently executing tasks in the terminal (e.g., ubuntu).
- pwd
  - **Explanation:** "Print Working Directory"—outputs the absolute path of the directory you are currently browsing.
- sudo apt update && sudo apt upgrade -y
  - **Explanation:** Refreshes local package tracking index metrics and upgrades all installed system software utilities to newer versions without prompting for manual confirmation.
- uname -r
  - **Explanation:** Outputs the specific running Linux kernel build version operating on the system.
- lsb_release -a
  - **Explanation:** Prints descriptive operating system details, version tracking releases, and distribution identity stats.
- mkdir vm-lab && cd vm-lab
  - **Explanation:** Generates a new file subdirectory workspace folder named vm-lab and moves into it.
- touch info.txt
  - **Explanation:** Spins up a completely blank empty text storage container template document called info.txt.
- echo "Running on AWS" > info.txt
  - **Explanation:** Overwrites the text inside info.txt with your custom string.
- cat info.txt
  - **Explanation:** Concatenates and prints the contents of the target text file directly onto the terminal console display grid.
- sudo apt install git -y
  - **Explanation:** Downloads and installs the Git version control package onto the virtual system engine.
- git --version
  - **Explanation:** Checks and confirms the current installed build release of the Git tool engine.
- sudo systemctl start nginx
  - **Explanation:** Instructs the system manager daemon to launch the background service process for the Nginx web server.
- sudo systemctl status nginx

- **Explanation:** Returns status information showing whether the Nginx web server is successfully running or halted.

4. Viva Q&A

- **Q: What is the main difference between a public IP address and a private IP address on a cloud VM?**
  - **A:** A public IP address is accessible over the global internet, allowing you to SSH into the VM or view its web pages. A private IP address is only visible and reachable within the cloud platform's internal network.
- **Q: Why must you configure an Security Group or Firewall rule to see an Nginx landing page?**
  - **A:** Cloud providers block all inbound traffic by default for security. You must explicitly open inbound TCP port 80 (HTTP) to let web browsers access your site.

—----------

Experiment 4: Installation of Docker and Running Basic Application Containers

1. Overview of the Experiment

This lab covers removing local runtime barriers by installing the open-source Docker containerization software layer directly on a Linux OS, checking its status, and deploying lightweight applications via isolated application containers.

2. How to Execute From Scratch

- Clean your dependencies, link public signature GPG keys, and install the structural Docker-CE package components.
- Start the underlying Docker service controller.
- Pull and run the validation hello-world test container directly from Docker Hub.
- Deploy a detached background Nginx server instance container mapped to target host ports.

3. Step-by-Step Command List

- sudo apt install apt-transport-https ca-certificates curl software-properties-common -y
  - **Explanation:** Installs baseline prerequisite OS layers to securely track, transfer, and download third-party tools over HTTPS channels.

- curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
  - **Explanation:** Downloads the verified public cryptography signature authorization key from Docker to validate incoming application downloads.
- sudo apt install docker-ce docker-ce-cli containerd.io -y
  - **Explanation:** Installs the core Docker engine application components, command line tools, and runtime systems.
- sudo systemctl start docker && sudo systemctl enable docker
  - **Explanation:** Launches the Docker daemon engine process immediately and ensures it automatically starts whenever the machine reboots.
- docker --version
  - **Explanation:** Returns the active binary software build version code string of your local engine installation.
- sudo docker run hello-world
  - **Explanation:** Searches for the hello-world image template locally. If it is not found, Docker pulls it from Docker Hub, launches it inside an isolated container, prints a confirmation message, and exits.
- sudo docker run -d -p 8080:80 nginx
  - **Explanation:** Launches an Nginx web server container in detached mode (-d) running continuously in the background, mapping the host's external port 8080 to the container's internal listening port 80 (-p).
- sudo docker ps
  - **Explanation:** Lists all currently active and running container instances, displaying their IDs, image names, and port mappings.
- sudo docker ps -a
  - **Explanation:** Lists every single container registered on the engine system infrastructure, including those that have exited or stopped.
- sudo docker rm <container_id>
  - **Explanation:** Deletes a specific stopped container instance from the host system disk memory storage entirely.

4. Viva Q&A

- **Q: What is the core difference between Virtualization (VMs) and Containerization (Docker)?**
  - **A:** A Virtual Machine requires a full hypervisor layer and packages an entire guest operating system alongside libraries, which consumes significant resources. A Docker container shares the host system's OS kernel and only packages the application code and dependencies, making it much more lightweight, fast, and portable.
- **Q: What does the command flag -p 8080:80 signify?**
  - **A:** It maps networks across barriers, binding external port 8080 of your host server directly to internal port 80 operating inside the isolated Docker container web service network.

—----------

Experiment 5: Custom Docker Images and Multi-Container Configurations (Docker Compose)

1. Overview of the Experiment

This lab focuses on writing a step-by-step Dockerfile blueprint to build custom application images and using Docker Compose to orchestrate multi-container setups (like a Python Flask web application communicating with a Redis database cache memory store).

2. How to Execute From Scratch

- Write your source code application file (app.py).
- Create a text file named Dockerfile detailing the base image layers, directory structures, dependencies, and startup command arguments.
- Build the custom application image and launch it.
- To step up to a multi-container architecture, define both services in a single docker-compose.yml file and start them simultaneously.

3. Step-by-Step Command List

- sudo docker build -t my-python-app .
  - **Explanation:** Reads the Dockerfile configuration in the current working directory (.) to build a custom immutable container image tagged (-t) as my-python-app.
- sudo docker images
  - **Explanation:** Scans local memory storage and lists all images currently downloaded or built on your local host system.
- sudo docker run -d -p 5000:5000 my-python-app
  - **Explanation:** Provisions and runs your custom Python application image in detached background mode, exposing the app web interface on host port 5000.
- sudo docker compose up --build
  - **Explanation:** Instructs Docker Compose to parse your docker-compose.yml file, automatically build any custom images, resolve service dependencies, and start all defined containers simultaneously within a shared network.

4. Sample Configuration Blueprint

Custom Dockerfile:

```
Dockerfile
FROM python:3.9-slim
WORKDIR /app
COPY app.py /app
RUN pip install flask redis
CMD ["python", "app.py"]
```

(Sources:)
Configuration Orchestration docker-compose.yml:

```
YAML
version: '3'
services:
  web:
    build: .
    ports:
      - "5000:5000"
  redis:
    image: "redis:alpine"
```

(Sources:)

5. Viva Q&A

- **Q: What is the relationship between a Dockerfile, a Docker Image, and a Docker Container?**
  - **A:** A Dockerfile is a text document containing instructions to build an image. A Docker Image is a read-only template file that acts as a blueprint. A Docker Container is a live, running instance instantiated from that image blueprint.
- **Q: Why use Docker Compose instead of launching multiple containers using separate docker run commands?**
  - **A:** Docker Compose lets you define and manage complex multi-container applications in a single declarative YAML file. With one command (docker compose up), it handles container spin-up order, configures internal networking so services can talk to each other, and simplifies cleanup.

—----------

Experiment 6: Infrastructure as Code (IaC) Automation Using Terraform

1. Overview of the Experiment

This lab focuses on using declarative Infrastructure as Code (IaC) with HashiCorp Terraform to automatically provision, modify, and teardown cloud infrastructure resources (VPCs, subnets, firewall structures, and compute engine instances) on public cloud service providers.

2. How to Execute From Scratch

- Download and extract the Terraform binary engine on your system, and add its path to your system's environment variables.
- Write an infrastructure declaration script named main.tf specifying your provider (e.g., AWS or GCP) and the exact infrastructure components you need.
- Initialize the workspace to download plugin providers, run a plan check, apply the configuration to create the live cloud resources, and run a destroy step to tear everything down.

3. Step-by-Step Command List

- terraform -version
  - **Explanation:** Verifies that the Terraform binary is correctly configured and prints the currently active build version.
- terraform init
  - **Explanation:** Initializes the working directory, scans your .tf configuration scripts, and automatically downloads the necessary cloud provider plugin drivers from the HashiCorp Registry.
- terraform validate
  - **Explanation:** Validates the structural syntax and internal logic consistency of your configuration files without connecting to real cloud APIs.
- terraform plan
  - **Explanation:** Generates a detailed preview execution blueprint showing exactly what resources Terraform will create, update, or destroy to match your code.
- terraform apply

- **Explanation:** Executes the deployment plan to provision live infrastructure components inside your public cloud platform account. It prompts for a manual confirmation (yes) before proceeding.
- terraform destroy
  - **Explanation:** Sweeps your cloud provider account and deletes every single resource managed by the local state tracking file to avoid ongoing cloud costs.

4. Core Resource Configuration Blueprint (main.tf)

```
Terraform
provider "aws" {
 region = "us-east-1"
}

resource "aws_vpc" "exp6_vpc" {
 cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "exp6_subnet" {
 vpc_id     = aws_vpc.exp6_vpc.id
 cidr_block = "10.0.1.0/24"
}

resource "aws_instance" "exp6_vm" {
 ami           = "ami-0440d3b780d96b29d"
 instance_type = "t2.micro"
 subnet_id     = aws_subnet.exp6_subnet.id
}
```

(Sources:)

5. Viva Q&A

- **Q: What is the purpose of the terraform.tfstate tracking file?**
  - **A:** The state file serves as Terraform's source of truth. It maps your real-world cloud resources to the resource declarations in your configuration code, tracks metadata, and determines what modifications are necessary during subsequent updates.
- **Q: What does the term "Declarative Language" mean in the context of Terraform?**
  - **A:** It means you only write code to describe the desired final state of your infrastructure (e.g., "I want an EC2 instance and a VPC"). You do not have to write step-by-step shell commands explaining *how* to build it; Terraform automatically figures out the correct creation order and dependencies.

—----------

Experiment 7: Configuration Management and Server Deployment Using Ansible

1. Overview of the Experiment

This lab covers automated configuration management using Ansible. It demonstrates how to establish an agentless control architecture using SSH to automatically provision user accounts, update packages, and deploy services across target server infrastructures.

2. How to Execute From Scratch

- Provision two separate cloud virtual machines: one to serve as your configuration **Control Node** and the other as the remote **Managed Host**.
- Install the central Ansible package on the Control Node.
- Generate an asymmetric RSA security key pair on your Control Node and copy the public key over to the Managed Host to enable passwordless authentication.
- Create a text tracking list named inventory detailing target IP routes, and write automated YAML deployment rule playbooks.

3. Step-by-Step Command List

- sudo apt install ansible -y
  - **Explanation:** Installs the core Ansible engine, execution binaries, and required automation core library scripts onto your control machine workspace.
- ansible --version
  - **Explanation:** Outputs the active software build release details of the Ansible deployment configuration layer.
- ssh-keygen -t rsa -b 2048
  - **Explanation:** Generates a secure 2048-bit RSA cryptographic public/private identity key pair signature to automate passwordless cross-server login workflows.
- cat ~/.ssh/id_rsa.pub
  - **Explanation:** Outputs your public cryptographic identity key string so you can copy and authorize it on target remote host environments.
- nano inventory
  - **Explanation:** Creates or opens an Ansible inventory configuration hostfile to organize your target infrastructure nodes by IP address or hostname.
- ansible -i inventory servers -m ping
  - **Explanation:** Uses the specified inventory tracking list file (-i) to target the [servers] group and runs the built-in ping module (-m) to verify SSH connectivity. A successful response returns a status of "ping": "pong".
- ansible-playbook -i inventory setup.yml

- **Explanation:** Tells Ansible to execute the step-by-step infrastructure automation tasks defined inside the setup.yml playbook against the target inventory hosts.

4. Playbook Configuration Blueprint (setup.yml)

```yaml
- name: Server configuration with roles
  hosts: servers
  become: yes
  tasks:
    - name: Create a new user
      user:
        name: devuser
        state: present
        shell: /bin/bash
    - name: Install basic software
      apt:
        name:
          - git
          - curl
        state: present
        update_cache: yes
```

(Sources:)

5. Viva Q&A

- **Q: What does it mean that Ansible is an "agentless" automation tool?**
  - **A:** Unlike other configuration management tools (like Puppet or Chef) that require you to install and maintain a dedicated background worker client program on every target machine, Ansible connects directly over standard SSH. It pushes raw code modules directly to the remote server, executes them, and removes them when finished.
- **Q: What does the parameter directive become: yes indicate inside a playbook script structure?**
  - **A:** It instructs Ansible to execute those specific playbook tasks with root administrator privileges (effectively running them via sudo), which is required for tasks like installing software packages or creating system user accounts.

—----------

Experiment 8: Building CI/CD Pipelines and Application Deployment via Jenkins Automation Server

1. Overview of the Experiment

This lab focuses on installing and using the open-source Jenkins automation engine to build an end-to-end continuous integration and continuous deployment (CI/CD) pipeline that automates pulling source code from a repository, running test logic check suites, and deploying applications.

2. How to Execute From Scratch

- Provision an ec2 server instance, ensuring that you open ingress security access channels on TCP port 8080 to access the web interface.
- Install Java (JDK 17) along with the official repository build packages for Jenkins, and start the system service manager.
- Retrieve the initial administrator unlock security password token string from the system log directories, complete the first-time setup wizard, and install the suggested core plugins (Git and Pipeline).
- Create a pipeline orchestration job using a structured Jenkinsfile configuration script.

3. Step-by-Step Command List

- sudo apt install openjdk-17-jdk -y
  - **Explanation:** Installs the Java Development Kit runtime environment, which is required because the Jenkins server architecture runs entirely on Java.
- sudo systemctl start jenkins && sudo systemctl enable jenkins
  - **Explanation:** Launches the background Jenkins integration engine process and ensures it starts automatically whenever the host server reboots.
- systemctl status jenkins
  - **Explanation:** Returns diagnostic runtime details to verify that your automated integration server is running successfully.
- sudo cat /var/lib/jenkins/secrets/initialAdminPassword
  - **Explanation:** Reads and prints the secure unique master password string required to unlock the Jenkins administrative web portal during initial setup.

- pip3 install flask pytest
  - **Explanation:** Installs your target application runtime framework dependencies along with the testing framework engine.
- pytest test_app.py
  - **Explanation:** Runs your automated unit test suite file to validate the logical integrity of your source code before triggering deployment.
- nohup python3 app.py &
  - **Explanation:** Launches your application script continuously in the background using the no-hangup system utility (nohup), allowing it to keep running even after your terminal session or build script finishes executing.

4. Declarative Pipeline Script Configuration Blueprint (Jenkinsfile)

```
pipeline {
    agent any
    stages {
        stage('Clone Source') {
            steps {
                git 'https://github.com/Arshad4786/jenkins.git'
            }
        }
        stage('Install Dependencies') {
            steps {
                sh 'pip3 install flask pytest'
            }
        }
        stage('Execute Logic Tests') {
            steps {
                sh 'echo "Running Tests..."'
                sh 'pytest || echo "No tests found"'
            }
        }
        stage('Deploy Staging') {
            steps {
                sh 'nohup python3 app.py &'
            }
        }
    }
}
```

(Sources:)

5. Viva Q&A

- **Q: What is the core difference between Continuous Integration (CI), Continuous Delivery (CD), and Continuous Deployment (CD)?**
  - **A: Continuous Integration** focuses on automatically building and running code verification tests every time a developer pushes changes to a shared repository to catch bugs early. **Continuous Delivery** extends this by automatically preparing your application code changes for eventual manual release to production. **Continuous Deployment** goes a step further by fully automating the pipeline so that every code change that passes your test suites is deployed straight to production without any manual human intervention.
- **Q: Why do we use nohup and the ampersand & symbol when running our application deployment script step in a Jenkins pipeline?**
  - **A:** By default, Jenkins tracks and manages all processes spawned during a build step stage. If you run a standard command like `python3 app.py`, the pipeline execution script will block and hang indefinitely while waiting for the web server process to close. Using `nohup ... &` detaches the web application process so it runs continuously in the system background, allowing Jenkins to successfully complete the build stage.